

NT Site Mirror

Turn any live website into editable code.

NT Site Mirror is a Claude Code / Codex skill that takes a live URL and produces a working local project: either a faithful static mirror of the deployed site, or a clean, component-based recreation ready for a rebrand or rebuild. It runs the same five-phase workflow every time — **Capture** → **Mirror or Recreate** → **Serve** → **Validate** → **Report** — so the result is reproducible, inspectable, and easy to hand off.

This guide covers installation, the two working modes, the five-phase workflow, and where to look when something needs special handling (WebGL/3D, video/audio, multi-route apps, late-loading content).

1. What's included

nt-site-mirror/	
SKILL.md	Entry point – the orchestrator agents read first
modes/	
static-mirror.md	Faithful local mirror workflow
editable-recreation.md	Clean, component-based rebuild workflow
modules/	
webgl-3d.md	Three.js / GLTF / Draco / HDR / KTX2
video-audio.md	Video, audio, HLS/DASH, Web Audio
runtime-assets.md	Lazy-loaded & late-loading content
multi-route.md	Route discovery, Next.js, Nuxt
templates/	
discovery.md	Capture findings template
animation-audit.md	Animation/interaction catalog
asset-preservation.md	Per-asset disposition tracker
validation-report.md	Cross-viewport QA template
scripts/	
capture.mjs	Phase 1 – crawl & download
mirror.mjs	Phase 2 – build static mirror
serve.mjs	Phase 3 – local static server
screenshot.mjs	Phase 4 – viewport screenshots
setup/	
claude-code-setup.md	
codex-setup.md	
docs/	
nt-site-mirror-guide.md (this file) + .pdf	

2. Choosing a mode

Static Mirror Mode

Preserves the deployed experience as closely as possible — same DOM, CSS, JS bundles, animations, media. Best for: offline copies, "swap a few words/images and keep everything else," preserving exact animation/video/WebGL behavior.

Editable Recreation Mode

Rebuilds the site as clean, component-based source (React/Next, Vite, or plain HTML/CSS/JS) using the capture as ground truth. Best for: rebrands, redesigns, maintainable component libraries, or when the original source is a mess.

Both modes run the same Capture, Serve, Validate, and Report phases — only Phase 2 differs. See `modes/static-mirror.md` and `modes/editable-recreation.md` for full details, and `SKILL.md` for the decision guide and hybrid fall-back strategy.

3. Setup

Claude Code

1. Copy `nt-site-mirror/` into `.claude/skills/nt-site-mirror/` (project) or `~/.claude/skills/nt-site-mirror/` (user-level).
2. Install script dependencies once:

```
cd .claude/skills/nt-site-mirror/scripts
npm install
npx playwright install chromium # if needed
```

3. Ask Claude to mirror or recreate a site — it will recognize the request and load `SKILL.md` automatically.

Full details, permissions, and troubleshooting: `setup/claude-code-setup.md`.

Codex

Codex doesn't have a dedicated skills directory — point it at the skill via `AGENTS.md`:

```
## NT Site Mirror
When asked to mirror, clone, or recreate a website as editable code, follow
skills/nt-site-mirror/SKILL.md.
```

Then install dependencies the same way (`npm install + npx playwright install chromium in scripts/`). Full details, including sandbox/network considerations for Codex cloud environments: `setup/codex-setup.md` .

4. The five-phase workflow

Phase 1 — Capture

```
cd nt-site-mirror/scripts
node capture.mjs https://example.com --out ../../nt-mirror/example/capture --max-pages 20
```

What it does:

- Discovers routes via `/sitemap.xml` , in-page links, and framework data (`__NEXT_DATA__` , `__NUXT__`)
- Loads each route in headless Chromium, downloads every network response (HTML, CSS, JS, images, fonts, video, audio, 3D assets, JSON/XHR)
- Scrolls the full page to trigger lazy-loaded and scroll-triggered content
- Saves rendered DOM snapshots, console logs, and framework state per route
- Writes `capture/manifest.json` and a pre-filled `capture/discovery.md`

Useful flags: `--max-pages` , `--settle-ms` (extra wait for late content like WebGL), `--scroll-pause-ms` , `--scroll-passes` , `--max-asset-mb` , `--routes a,b,c` (seed extra routes).

After capture, fill in `capture/discovery.md` — it's pre-populated with the route list and asset inventory; add notes on animations, WebGL/3D, video/audio, runtime assets, and external dependencies using the linked module/template files.

Phase 2 — Mirror or Recreate

Static Mirror Mode:

```
node mirror.mjs ../../nt-mirror/example/capture ../../nt-mirror/example/site
```

Copies downloaded assets into `site/` (same-origin assets keep their path; cross-origin assets go under `site/_cdn/<host>/...`), rewrites `href / src / srcset /CSS url()` references to local paths, and writes one `index.html` per captured route — automatically using the rendered DOM snapshot instead of the raw source HTML when the source looks like an empty SPA shell. Flags potential service-worker registrations and hardcoded absolute-URL references in JS for manual review.

Editable Recreation Mode: Follow `modes/editable-recreation.md` — extract design tokens from captured CSS, componentize sections from the rendered DOM, recreate animations per `templates/animation-audit.md` , and apply the requested rebrand/redesign changes.

Track every asset's disposition in `templates/asset-preservation.md` as you go.

Phase 3 — Serve

```
node serve.mjs ../../nt-mirror/example/site --port 4173
# For SPAs with client-side routing:
node serve.mjs ../../nt-mirror/example/site --port 4173 --spa-fallback index.html
```

Static file server with correct MIME types for fonts, video/audio, and WebGL/3D formats (`.glb` , `.gltf` , `.hdr` , `.ktx2` , `.drc` , etc.). For framework-based recreations, use the project's own `npm run dev` / `npm run build && npm run preview` instead.

Phase 4 — Validate

```
node screenshot.mjs http://localhost:4173 --out ../../nt-mirror/example/validation --routes /,
```

Captures Desktop (1440×900), Tablet (768×1024), and Mobile (390×844) screenshots per route, plus optional console-error/failed-request reports. Compare against screenshots of the live site and walk through `templates/validation-report.md` : visual diffs, console errors, broken links, and an interaction/animation spot-check informed by the module checklists (WebGL, video/audio).

Fix anything that doesn't match before moving on — don't report known, fixable issues as "done."

Phase 5 — Report

Write `REPORT.md` summarizing: what was captured, which mode was used and why, fidelity results, blocked/missing assets (with original URLs), excluded external dependencies, exact run commands, and suggested next steps. Use `templates/asset-preservation.md` and `templates/validation-report.md` as inputs.

5. Special content — quick reference

If the site has...	Read
<code><canvas></code> w/ WebGL, <code>.glb</code> / <code>.gltf</code> / <code>.hdr</code> / <code>.ktx2</code> / Draco/Spline	<code>modules/webgl-3d.md</code>
Background video, HLS/DASH streams, Web Audio SFX	<code>modules/video-audio.md</code>
Lazy-loaded images, code-split chunks, CSS-in-JS, A/B tests	<code>modules/runtime-assets.md</code>
Many routes, Next.js/Nuxt, sitemaps, dynamic routes	<code>modules/multi-route.md</code>

6. Working directory layout

```
nt-mirror/<site-slug>/
  capture/          raw captured HTML, assets, manifest.json, discovery.md
  site/             the rebuilt project (mirror or recreation)
  validation/       before/after screenshots + console reports
  REPORT.md
```

7. Troubleshooting

Symptom	Cause	Fix
<code>browserType.launch: Executable doesn't exist</code>	Playwright/Chromium version mismatch	<code>npx playwright install chromium</code> , or pin playwright in <code>scripts/package.json</code> to an installed revision
<code>net::ERR_CERT_AUTHORITY_INVALID</code>	Self-signed/internal cert	Already handled (<code>ignoreHTTPSErrors: true</code>)
Only the entry route is captured	No sitemap, unusual link markup	Pass <code>--routes /a,/b,/c</code>
Blank canvas / missing hero animation	Late-loading WebGL/video not finished	Increase <code>--settle-ms</code> , <code>--scroll-pause-ms</code> , <code>--scroll-passes</code> ; see <code>modules/runtime-assets.md</code> and <code>modules/webgl-3d.md</code>
Second click after load 404s on a Next.js mirror	<code>_next/data/<buildId>/<route>.json</code> wasn't captured for that route	Expected if only a subset of routes was captured — note in report, or re-run capture including that route
<code>EADDRINUSE</code> from <code>serve.mjs</code>	Port in use	<code>--port <other-port></code>

8. Support

This skill is a set of plain-text instructions and Node.js scripts — no external service or license server is involved. To extend it for a specific site (e.g., a custom "enter site" interaction for a 3D intro), edit `scripts/capture.mjs` directly; the script is intentionally short and commented at each step.

For questions about Claude Code itself (skills, hooks, permissions), see Anthropic's Claude Code documentation. For Codex configuration (sandbox modes, `AGENTS.md`, network allowlists), see your

Codex environment's documentation.